

Semantic PDF Transformation Platform

Architecture RFC — Technical Design Document

RFC-001

June 2026

Contents

1	Executive Summary	4
1.1	Technical Thesis	4
1.2	Scope	4
2	Deep Dive: Why PDF Editing Is Hard	5
2.1	2.1 The PDF Object Model	5
2.2	2.2 Content Stream Example	5
2.3	2.3 Font Encodings and Glyph Mapping	5
2.4	2.4 Why Redraw-Based Editing Fails	6
3	Existing Approaches	8
3.1	3.1 PyMuPDF (fitz)	8
3.2	3.2 PDF → DOCX → PDF Round-Trip	8
3.3	3.3 Adobe Acrobat / PDFTron / iText	9
3.4	3.4 Docling / Unstructured / Azure Document Intelligence	9
3.5	3.5 Proposed System: Semantic Transformation	9
4	PDF Reconstruction Layer	11
4.1	4.1 Content Stream Parser	11
4.1.1	4.1.1 Responsibilities	11
4.1.2	4.1.2 Operator Handling	11
4.1.3	4.1.3 Parser Algorithm (Simplified)	12
4.1.4	4.1.4 Glyph Position Calculation	12
4.2	4.2 Font Graph	13
4.2.1	4.2.1 Data Structure	13
4.2.2	4.2.2 Unicode Resolution Algorithm	13
4.2.3	4.2.3 ToUnicode CMap Parsing	14
4.3	4.3 Glyph Position Graph	14
4.3.1	4.3.1 Rotation Detection	14
4.4	4.4 XObject Traversal	14
4.4.1	4.4.1 Traversal Algorithm	15
4.4.2	4.4.2 Nesting Depth	15
5	Text Fragmentation Problem	16
5.1	5.1 Problem Statement	16
5.2	5.2 Fragmentation Patterns by Generator	16
5.2.1	5.2.1 Microsoft Word → PDF	16
5.2.2	5.2.2 LaTeX → PDF (pdflatex)	16
5.2.3	5.2.3 LaTeX → PDF (lualatex/xelatex)	16
5.2.4	5.2.4 Adobe InDesign → PDF	16
5.2.5	5.2.5 Chrome/Web → PDF	17

5.3	5.3 Visual-to-Logical Reconstruction	17
5.3.1	5.3.1 Global Text Layer Construction	17
5.3.2	5.3.2 Y-Clustering Algorithm	17
5.3.3	5.3.3 Token Clustering Algorithm	18
5.3.4	5.3.4 Edge Cases	18
6	Subset Font Problem	19
6.1	6.1 Problem Statement	19
6.2	6.2 CID Mapping Deep Dive	19
6.3	6.3 Font Expansion Strategy (FontTools)	19
6.3.1	6.3.1 Algorithm: Subset Font Expansion	19
6.3.2	6.3.2 Locating the Full Font	20
6.4	6.4 Why Most Editors Avoid This Problem	21
6.5	6.5 Hybrid Strategy (Recommended)	21
7	Semantic Layer	22
7.1	7.1 Document Object Graph	22
7.2	7.2 Entity Definition Schema	22
7.3	7.3 Entity Types Catalog	22
7.4	7.4 Entity Extraction Pipeline	23
7.5	7.5 Confidence Scoring	23
8	Semantic Editing Engine	25
8.1	8.1 Core Operation	25
8.2	8.2 Edit Pipeline	25
8.2.1	8.2.1 Step 1: Entity Resolution	25
8.2.2	8.2.2 Step 2: PDF Object Resolution	25
8.2.3	8.2.3 Step 3: Glyph Validation	25
8.2.4	8.2.4 Step 4: Content Stream Mutation	26
8.2.5	8.2.5 Step 5: PDF Structural Update	26
8.2.6	8.2.6 Step 6: Validation	26
8.3	8.3 Edit Granularity Levels	26
9	Competition Analysis	28
9.1	9.1 Technical Comparison Matrix	28
9.2	9.2 Market Positioning	29
10	Failure Modes	30
10.1	10.1 Missing Glyphs	30
10.2	10.2 Broken or Missing ToUnicode CMaps	30
10.3	10.3 Image-Only PDFs (Scanned Documents)	30
10.4	10.4 Rotated Text	30
10.5	10.5 Tables	31
10.6	10.6 Watermarks	31
10.7	10.7 Headers and Footers	31
10.8	10.8 Multi-Column Layouts	31
11	Evaluation Framework	33
11.1	11.1 Core Metrics	33
11.1.1	11.1.1 Text Reconstruction Accuracy (TRA)	33
11.1.2	11.1.2 Entity Recall (ER)	33
11.1.3	11.1.3 Layout Preservation Score (LPS)	33
11.1.4	11.1.4 Edit Success Rate (ESR)	33
11.1.5	11.1.5 Font Preservation Score (FPS)	34
11.2	11.2 Benchmark Dataset Design	34
12	Engineering Roadmap	35
12.1	12.1 Phase 1: Reconstruction Engine (Months 1-3)	35
12.1.1	12.1.1 Deliverables:	35

12.1.2	Key Risks:	35
12.2	Phase 2: Layout Graph (Months 3-5)	35
12.2.1	Deliverables:	35
12.2.2	Key Risks:	35
12.3	Phase 3: Semantic Layer + Editing (Months 5-9)	35
12.3.1	Deliverables:	36
12.3.2	Key Risks:	36
12.4	Phase 4: API Platform (Months 9-12)	36
12.4.1	Deliverables:	36
12.5	Phase 5: Enterprise Workflows (Months 12-18)	36
12.5.1	Deliverables:	36
13	Open Questions	37
13.1	13.1 Subset Font Expansion Limits	37
13.2	13.2 Semantic Confidence Measurement	37
13.3	13.3 Layout Graph Generalization	37
13.4	13.4 OCR Integration Depth	37
13.5	13.5 Performance at Scale	37
13.6	13.6 Security Considerations	37

1 Executive Summary

1.1 Technical Thesis

Existing systems in the PDF ecosystem fall into exactly one of two mutually exclusive categories: **document understanding** systems that extract meaning but cannot modify, and **document editing** systems that modify but have no semantic awareness of what they are changing.

This document proposes a third category: a **Semantic PDF Transformation Platform** that establishes a reversible, bidirectional mapping between semantic entities (person names, dates, invoice numbers, contract parties) and the original PDF objects (content streams, glyphs, font encodings, positioning commands) that render them. This mapping enables semantic-level modifications — “change the applicant name from SANKET to TEST” — while preserving 100% of the original visual fidelity for every untouched portion of the document.

The core innovation is not OCR. It is not a new PDF rendering engine. It is the **entity-to-object reverse mapping** that allows editing at the semantic layer while mutating only the specific PDF primitives that correspond to the target entity, leaving the rest of the document’s byte-level rendering untouched.

1.2 Scope

This RFC covers the full architecture: problem decomposition, PDF internals, content stream parsing, font graph construction, glyph resolution, layout reconstruction, text fragmentation handling, the subset font expansion problem, the semantic entity layer, the mutation engine, competition analysis, failure modes, an evaluation framework, an engineering roadmap, and open research questions.

2 Deep Dive: Why PDF Editing Is Hard

2.1 2.1 The PDF Object Model

PDF is a page description language, not a document format. Its object model reflects its origin as a print-ready output format where the primary concern is “where does ink go on paper” rather than “what words are on this page.”

Catalog

```

+--- Pages (page tree)
    +--- Page
        +--- MediaBox (page dimensions)
        +--- Resources
            |   +--- Font (font dictionaries)
            |   +--- XObject (reusable forms, images)
            |   +--- ExtGState (transparency, blend modes)
        +--- Contents
            +--- Content Stream (sequence of operators)

```

A Page object does not contain text. It contains a reference to a Content Stream. The Content Stream is a sequence of stack-based operators that draw glyphs, lines, images, and shading onto a virtual canvas. Text is not stored as characters; it is stored as glyph indices into font programs, positioned by transformation matrices.

2.2 2.2 Content Stream Example

A minimal “Hello World” content stream:

```

BT                                % Begin Text
/F1 12 Tf                        % Set font F1 at 12pt
72 720 Td                        % Move text position to (72, 720)
(Hello World) Tj                 % Show text string
ET                                % End Text

```

This appears straightforward. But in practice, real-world PDF generators produce streams like this:

```

BT
/F1 12 Tf
1 0 0 1 72 720 Tm
[(S) 5 (A) 3 (N) 2 (K) -2 (E) 3 (T)] TJ
ET

```

The TJ operator accepts an array of strings and numbers. Strings are glyph sequences; numbers are kerning adjustments (in thousandths of an em). The text “SANKET” is fragmented across five string elements with spacing adjustments between them. There is no guarantee that a word fits inside a single Tj or TJ call. A single word may span multiple text blocks, multiple content streams, or even multiple XObjects.

2.3 2.3 Font Encodings and Glyph Mapping

When a PDF says (A) Tj, the byte 0x41 inside the string does not necessarily mean the character ‘A’. It is an index into the font’s encoding vector. The font may use:

Encoding Type	Description
WinAnsiEncoding	Standard 8-bit encoding, byte 0x41 = ‘A’
MacRomanEncoding	Apple’s 8-bit encoding

Encoding Type	Description
Custom Encoding	Differences array mapping byte values to glyph names
Identity-H (CID)	2-byte CID values, requires CMap for Unicode mapping
Identity-V	Vertical writing variant of Identity-H

For CID fonts (common in East Asian PDFs and many modern documents), the glyph identifier is a 2-byte CID. To map it to Unicode, the system must:

1. Read the font's **ToUnicode** CMap stream (if present)
2. Fall back to the font's Encoding dictionary
3. As a last resort, use the glyph name from the font program and map it via Adobe Glyph List

Each of these steps can fail independently. A PDF may have a **ToUnicode** CMap that maps only a subset of glyphs, an Encoding dictionary that uses non-standard names, or a font program where glyph names do not follow any standard convention.

2.4 2.4 Why Redraw-Based Editing Fails

Consider a real-world resume PDF. The name “SANKET” appears in the header. A naive redraw-based editor:

1. Extracts the text “SANKET” via OCR or text extraction
2. Locates the bounding box containing “SANKET”
3. Draws a white rectangle over the bounding box (redaction)
4. Draws “TEST” in a fallback font at the same position
5. Saves the PDF

Expected output:

TEST

Actual observed output (from real testing):

```
TEST
  TEST
    TEST
```

Root cause analysis:

The text “SANKET” was not a single text operator. It was rendered as:

```
BT
/F1 10 Tf
72 720 Td
[(S) 3 (A) 2 (N)] TJ
ET
BT
/F1 10 Tf
200 720 Td
[(K) 2 (E) 2 (T)] TJ
ET
```

The redaction rectangle covered the visual bounding box, but the text was rendered by two separate BT/ET blocks. The second block's glyphs were not fully covered by the whiteout rectangle, causing partial rendering of both old and new text. Additionally, the replacement font's glyph widths differed from the original, so "TEST" occupied fewer horizontal units than "SANKET", leaving the original glyphs partially visible on the right side.

The failure modes cascade:

1. **Fragmented operators:** Text split across multiple TJ/Tj calls, BT/ET blocks, or even across multiple content streams for the same page
2. **Multiple matching bounding boxes:** The string "SANKET" may appear in the header, footer, body text, and a watermark — which one does the user mean?
3. **Redraw detachment:** The newly drawn text has no relationship to the original PDF objects, so subsequent edits must redraw again (each cycle degrades fidelity)
4. **Font mismatch:** The replacement font has different glyph metrics, causing text reflow, clipping, or overlap with adjacent content
5. **Absence of semantic resolution:** The system doesn't know that this specific occurrence of "SANKET" is a `Person.Name` entity vs. incidental text

3 Existing Approaches

3.1 3.1 PyMuPDF (fitz)

Architecture: PyMuPDF wraps the MuPDF rendering engine. It provides text extraction via `page.get_text("words")` and redaction via `page.add_redact_annot()` followed by `page.apply_redactions()`.

Redaction flow:

```
page.add_redact_annot(bbox)
page.apply_redactions()      # MuPDF redraws affected area
page.insert_text(point, "TEST", fontname="helv")
```

Pros:

- Fast, C-based rendering engine
- Handles most PDF variants
- Good text extraction quality
- Redaction is a built-in operation

Cons:

- Redaction + `insert_text` is a destructive redraw: MuPDF regenerates the content stream for the affected region
- No semantic entity concept — redaction is purely geometric (bounding box based)
- Inserted text uses a new font, not the original — font fidelity is lost
- Cannot target a *specific semantic entity* across multiple occurrences
- Multi-block text (fragmented operators) requires multiple redaction regions, which the user must manually specify
- No reverse mapping from text back to PDF objects

3.2 3.2 PDF → DOCX → PDF Round-Trip

Architecture: Convert the PDF to an editable format (DOCX, HTML, or plain text), modify it, then regenerate the PDF.

Tools: `pdf2docx`, `LibreOffice --headless --convert-to`, `pandoc`

Pros:

- Leverages mature document editors (Word, LibreOffice)
- Full editing capabilities in the intermediate format
- Works with any PDF that can be converted

Cons:

- **Total fidelity loss:** The regenerated PDF is a completely new document. Fonts, margins, spacing, image placement, table formatting, headers/footers — everything is reconstructed from scratch
- **Round-trip instability:** PDF → DOCX → PDF is not idempotent. Running it twice produces a different result
- **Silent corruption:** Page breaks shift, tables reflow, fonts are substituted without warning
- **Non-text content:** Vector graphics, watermarks, stamps, and annotations are often dropped or rasterized
- **Not suitable for single-field edits:** Regenerating 20 pages to change one name is architecturally excessive

3.3 3.3 Adobe Acrobat / PDFTron / iText

Architecture: These are professional PDF SDKs that operate at the PDF object level. They can manipulate content streams, fonts, and page descriptions directly.

Pros:

- Full access to PDF internals
- Can perform precise, targeted edits to content streams
- Font embedding and subset management
- Industry-standard, well-tested

Cons:

- **No semantic layer:** They operate on PDF objects, not entities. “Change the name” requires the developer to locate the correct content stream, identify the correct text operators, and manually compute new glyph positions
- **Proprietary / expensive:** Adobe Acrobat SDK and PDFTron require commercial licenses. iText is AGPL (viral) or commercial
- **Limited automation APIs:** Acrobat’s JavaScript API is restricted; SDKs require deep PDF expertise
- **Developer-unfriendly:** Building a “change all occurrences of Person.Name” feature on top of these SDKs requires reimplementing most of the parsing and layout logic anyway

3.4 3.4 Docling / Unstructured / Azure Document Intelligence

Architecture: These are document understanding platforms. They ingest PDFs and images and output structured JSON with text, tables, and layout information.

Pros:

- Excellent text extraction quality (especially Azure’s OCR)
- Table extraction and structure recognition
- Reading order detection
- Document classification

Cons:

- **Read-only:** They extract information but have no write path back to the original PDF
- **No PDF object mapping:** The extracted text has no link to the original PDF’s content streams, fonts, or glyphs
- **Output is a new artifact:** The JSON structure is a derived representation, not a reversible transformation of the original

3.5 3.5 Proposed System: Semantic Transformation

Core insight: Combine the PDF-object-level access of PDFTron/iText with the semantic understanding of Docling/Unstructured, bridged by a bidirectional entity-to-object mapping.

Capability	PyMuPDF	PDF→DOCX→PDF	PDF/iText	Docling	Proposed
Text extraction	[x]	[x]	[x]	[x]	[x]
PDF object access	Partial	[]	[x]	[]	[x]
Targeted editing	[]	[]	[x]	[]	[x]
Semantic entities	[]	[]	[]	[x]	[x]
Font fidelity preservation	[]	[]	[x]	N/A	[x]

Capability	PyMuPDF	PDF→DOCX→PDF	Adobe/iText	Docling	Proposed
Entity→PDF reverse map	[]	[]	[]	[]	[x]
Automation- friendly API	Partial	[x]	Partial	[x]	[x]
Open source	[x]	[x]	Partial	[x]	[x]

4 PDF Reconstruction Layer

The reconstruction layer is the foundation. Its job is to parse a PDF and construct a complete, queryable model of every text element on every page, with bidirectional links between Unicode text, glyphs, fonts, positioning, and the original PDF objects.

4.1 4.1 Content Stream Parser

4.1.1 4.1.1 Responsibilities

The parser ingests raw content streams (decompressed from FlateDecode/ASCII85Decode) and produces a canonical text layer.

Input: Raw content stream bytes (post-decompression) **Output:** A list of `TextFragment` structures, each containing:

```
TextFragment {
    page_id: int
    stream_id: int
    object_ref: str          # e.g. "12 0 R"
    unicode_text: str        # "SANKET"
    glyph_ids: list[int]     # [54, 33, 47, 40, 38, 52] (CIDs or char codes)
    positions: list[(float, float)] # (x, y) for each glyph
    font_key: str            # e.g. "/F1"
    font_size: float
    transform_matrix: [float; 6] # Tm value
    rendering_mode: int       # 0=fill, 1=stroke, 2=fill+stroke, etc.
    color: (float, float, float) # fill color in deviceRGB
}
```

4.1.2 4.1.2 Operator Handling

The parser implements handlers for these PDF text operators:

Operator	Name	Action
BT	Begin Text	Start a new text block; reset text matrix
ET	End Text	End the text block
Tf	Set Font	Set current font and size
Td	Move Text	Move to new line (offset)
TD	Move Text + Leading	Move and set leading
Tm	Set Text Matrix	Set transformation matrix
T*	Next Line	Move to next line
Tj	Show Text	Draw a string
TJ	Show Text Array	Draw array of strings with kerning
,	Next Line + Show	T* + Tj
“	Set Leading + Move + Show	Set word/char spacing + T* + Tj
Ts	Set Text Rise	Superscript/subscript offset
Tc	Set Char Spacing	Additional inter-character spacing
Tw	Set Word Spacing	Additional inter-word spacing
Tz	Set Horizontal Scaling	Scale glyph widths
Tr	Set Rendering Mode	Fill, stroke, fill+stroke, invisible, etc.

4.1.3 4.1.3 Parser Algorithm (Simplified)

```
def parse_content_stream(stream_bytes, resources, page_id):
    state = TextState()
    fragments = []

    for op, operands in tokenize(stream_bytes):
        match op:
            case b"BT":
                state = TextState() # Reset text state
            case b"ET":
                flush_text_buffer(state, fragments)
            case b" Tf":
                font_key, size = operands
                state.font = resources.Font[font_key]
                state.font_size = size
            case b" Tm":
                state.text_matrix = operands # [a,b,c,d,e,f]
            case b" Td" | b" TD":
                tx, ty = operands
                state.text_matrix[4] += tx
                state.text_matrix[5] += ty
            case b" Tj":
                text = operands[0]
                process_text(text, state, fragments)
            case b" TJ":
                for item in operands[0]:
                    if isinstance(item, bytes):
                        process_text(item, state, fragments)
                    else:
                        state.text_matrix[4] -= item/1000 * state.font_size
                        # Apply kerning adjustment
            case b" '":
                state.text_matrix[4] = 0 # Carriage return
                state.text_matrix[5] -= state.leading
                process_text(operands[0], state, fragments)
            case b" Ts":
                state.text_rise = operands[0]
            case b" Tc":
                state.char_spacing = operands[0]
            case b" Tw":
                state.word_spacing = operands[0]
            case b" Tz":
                state.horizontal_scaling = operands[0] / 100.0
            case b" Tr":
                state.rendering_mode = operands[0]

    return fragments
```

4.1.4 4.1.4 Glyph Position Calculation

For each glyph in a text string, the rendered position is:

```
x_render = tm[0]*x + tm[2]*y + tm[4]
y_render = tm[1]*x + tm[3]*y + tm[5]
```

where `tm` is the text matrix and (x, y) is the glyph's horizontal displacement computed as:

```
displacement = (glyph_width * font_size * horizontal_scaling / 1000.0)
               + char_spacing
               + (word_spacing if glyph is space else 0)
```

If the text rise (Ts) is nonzero:

```
y_render += text_rise
```

4.2 4.2 Font Graph

The font graph maintains bidirectional mappings between Unicode code points, glyph identifiers (CIDs or character codes), glyph names, and the font program itself.

4.2.1 4.2.1 Data Structure

```
type FontNode struct {
    FontID          string           // PDF font reference, e.g. "R12"
    FontName        string           // BaseFont or descendant font name
    Subtype         string           // Type0, Type1, TrueType, CIDFontType0, CIDFontType2
    Encoding        *Encoding        // WinAnsiEncoding, MacRomanEncoding, or custom
    ToUnicode       *CMap            // ToUnicode CMap stream (CID → Unicode)
    CIDToGID        map[int]int      // CID → internal GID (for CID fonts)
    GIDToUnicode    map[int]string   // GID → Unicode (from cmap table for TrueType)
    UnicodeToCID    map[rune]int     // Reverse: Unicode → CID
    GlyphWidths     map[int]float    // Glyph widths in thousandths of em
    FontDescriptor  *FontDescriptor // Font metadata
    FontFile        []byte          // Embedded font program (if present)
    IsSubset        bool            // True if the font was subset-embedded
    DefaultWidth    float           // Default glyph width
}
```

4.2.2 4.2.2 Unicode Resolution Algorithm

Given a glyph identifier (either a 1-byte char code or 2-byte CID) and a font node, resolve to Unicode:

```
def resolve_to_unicode(font_node, glyph_id):
    if font_node.ToUnicode is not None:
        unicode_val = font_node.ToUnicode.lookup(glyph_id)
        if unicode_val is not None:
            return chr(unicode_val)

    if font_node.Encoding is not None:
        glyph_name = font_node.Encoding.lookup(glyph_id)
        if glyph_name is not None:
            return adobe_glyph_list.to_unicode(glyph_name)

    if font_node.Subtype in ("TrueType", "CIDFontType2"):
        unicode_val = font_node.GIDToUnicode.get(glyph_id)
        if unicode_val is not None:
            return unicode_val

    return REPLACEMENT_CHARACTER # U+FFFD
```

4.2.3 ToUnicode CMap Parsing

ToUnicode CMaps use a subset of the PostScript CMap format. A typical entry:

```
<0001> <0020>      # CID 1 → U+0020 (space)
<0002> <0041>      # CID 2 → U+0041 (A)
<0003> <0042>      # CID 3 → U+0042 (B)
beginbfrange
<0004> <001A> <0043>  # CIDs 4-26 → U+0043 through U+005F
endbfrange
```

The parser must handle **bfchar** (single mappings), **bfrange** (range mappings), and the less common **bfrange** with source/destination ranges of different sizes.

4.3 4.3 Glyph Position Graph

The position graph records the exact coordinates of every glyph on every page.

```
type PositionNode struct {
    PageID      int
    GlyphID     int           // Internal glyph identifier
    X           float64      // X coordinate in PDF user space
    Y           float64      // Y coordinate in PDF user space
    Width       float64      // Glyph width in user space units
    Height      float64      // Approximate glyph height
    FontSize    float64
    Rotation    float64      // 0, 90, 180, 270 (extracted from text matrix)
}
```

4.3.1 4.3.1 Rotation Detection

Rotation is extracted from the text matrix T_m :

```
Tm = [a, b, c, d, e, f]
rotation_rad = atan2(b, a)
rotation_deg = rotation_rad * 180 / PI
```

Common rotation values and their meanings:

a	b	c	d	Rotation	Meaning
1	0	0	1	0°	Normal horizontal text
-1	0	0	-1	180°	Upside-down text
0	1	-1	0	90°	Rotated 90° clockwise
0	-1	1	0	270° (-90°)	Rotated 90° counterclockwise

4.4 4.4 XObject Traversal

Page content often lives in Form XObjects — reusable page components like headers, footers, watermarks, and logos. The parser must recursively descend into XObjects.

4.4.1 4.4.1 Traversal Algorithm

```
def parse_page(page_dict, resources, page_id):
    all_fragments = []

    # Parse the page's direct content stream(s)
    contents = page_dict.get("Contents", [])
    if not isinstance(contents, list):
        contents = [contents]

    for stream_ref in contents:
        stream = resolve_stream(stream_ref)
        fragments = parse_content_stream(stream, resources, page_id)
        all_fragments.extend(fragments)

    # Check stream for Do (XObject) operators
    xobj_refs = extract_xobject_refs(stream)
    for ref in xobj_refs:
        xobj = resolve_stream(ref)
        xobj_resources = xobj.dict.get("Resources", resources)
        xobj_fragments = parse_content_stream(xobj, xobj_resources, page_id)
        all_fragments.extend(xobj_fragments)

    return all_fragments
```

4.4.2 4.4.2 Nesting Depth

In practice, most PDFs nest no deeper than 2-3 levels. However, the parser should support arbitrary depth with a configurable max depth (default: 10) and a cycle detection mechanism (track visited object references).

5 Text Fragmentation Problem

5.1 5.1 Problem Statement

In a visual PDF viewer, the word “SANKET” appears as five contiguous letters. In the content stream, it may be stored as:

```
[(S) 5 (A) 3 (N) 2 (K) -2 (E) 3 (T)] TJ
```

Or across multiple blocks:

```
BT /F1 10 Tf 72 720 Td (SAN) Tj ET
BT /F1 10 Tf 100 720 Td (KET) Tj ET
```

Or interleaved with unrelated content:

```
BT /F1 10 Tf 72 720 Td (SAN) Tj ET
% ... other drawing commands ...
BT /F1 10 Tf 100 720 Td (KET) Tj ET
```

This fragmentation is not a bug; it is how PDF generators optimize glyph placement. Every PDF generator fragments text differently:

5.2 5.2 Fragmentation Patterns by Generator

5.2.1 Microsoft Word → PDF

```
[(T) 3 (h) 2 (i) -1 (s) 5 ( ) 2 (i) 2 (s)] TJ
```

Word fragments at character boundaries with precise kerning values between every character to match its internal layout engine’s output.

5.2.2 LaTeX → PDF (pdflatex)

```
(This is a sentence.) Tj
```

pdflatex typically emits whole words or phrases in a single Tj call. Fragmentation occurs at ligature boundaries (fi, fl, ff) and at font changes within a line.

5.2.3 LaTeX → PDF (lualatex/xelatex)

```
[(T) 2 (h) 1 (i) 3 (s)] TJ
```

Similar to Word, these engines use per-character positioning with small kerning adjustments.

5.2.4 Adobe InDesign → PDF

```
<0048 0065 006C 006C 006F> Tj
```

InDesign typically uses hex-encoded glyph sequences with Identity-H encoding, often emitting entire words or lines as single strings.

5.2.5 Chrome/Web → PDF

```
[(S) -2 (A) -1 (N) 0 (K) -1 (E) 2 (T)] TJ
```

Browser PDF engines fragment aggressively, sometimes emitting individual characters with per-character spacing derived from the HTML/CSS layout.

5.3 5.3 Visual-to-Logical Reconstruction

The goal is to reconstruct the logical reading order from fragmented glyph operators.

5.3.1 5.3.1 Global Text Layer Construction

```
def build_global_text_layer(page_fragments):
    # Step 1: Collect all glyphs with positions
    glyphs = []
    for frag in page_fragments:
        for i, unicode_char in enumerate(frag.unicode_text):
            if unicode_char == " ":
                continue
            glyphs.append(Glyph(
                char=unicode_char,
                x=frag.positions[i][0],
                y=frag.positions[i][1],
                width=frag.glyph_widths[i],
                height=frag.font_size,
                fragment_ref=frag,
                glyph_index=i
            ))

    # Step 2: Group glyphs into lines by Y-coordinate proximity
    tolerance = 0.5 # points
    lines = cluster_by_y(glyphs, tolerance)

    # Step 3: Within each line, sort by X-coordinate
    for line in lines:
        line.sort(key=lambda g: g.x)

    # Step 4: Group into tokens (words) by horizontal gaps
    for line in lines:
        tokens = cluster_into_words(line, gap_threshold=3.0)

    # Step 5: Assign reading order (top-to-bottom lines)
    lines.sort(key=lambda l: -l[0].y) # PDF y-axis goes up

    return lines
```

5.3.2 5.3.2 Y-Clustering Algorithm

```
def cluster_by_y(glyphs, tolerance):
    # Sort glyphs by Y position
    sorted_glyphs = sorted(glyphs, key=lambda g: -g.y)
    lines = []
    current_line = [sorted_glyphs[0]]

    for g in sorted_glyphs[1:]:
```

```

    if abs(g.y - current_line[-1].y) < tolerance:
        current_line.append(g)
    else:
        lines.append(current_line)
        current_line = [g]
    lines.append(current_line)
    return lines

```

Complexity: $O(n \log n)$ due to sorting, where n is the number of glyphs on a page. Typical page: 500-5000 glyphs. Runtime: negligible.

5.3.3 5.3.3 Token Clustering Algorithm

```

def cluster_into_words(line_glyphs, gap_threshold):
    tokens = []
    current_token = [line_glyphs[0]]

    for i in range(1, len(line_glyphs)):
        gap = line_glyphs[i].x - (line_glyphs[i-1].x + line_glyphs[i-1].width)
        if gap > gap_threshold:
            tokens.append(Token(glyphs=current_token))
            current_token = [line_glyphs[i]]
        else:
            current_token.append(line_glyphs[i])
    tokens.append(Token(glyphs=current_token))
    return tokens

```

5.3.4 5.3.4 Edge Cases

1. **Subscript/superscript:** Glyphs with different Y positions within what should be the same word. Detected via text rise (Ts) operator or small Y differences with baseline knowledge.
2. **Rotated text:** The clustering algorithm must account for rotation. For rotated text, the “reading axis” is not the X axis. The algorithm detects rotation from the text matrix and applies the appropriate clustering axis.
3. **Right-to-left text:** Arabic and Hebrew require RTL glyph ordering within tokens. Detected via the font’s writing direction or Unicode bidi algorithm.
4. **Diacritics and combining marks:** Combining characters (accents, vowel marks) share the same or nearly identical X position as their base character. They must be merged with their base character, not treated as separate tokens.
5. **Vertical text:** Japanese vertical text requires clustering by Y (top to bottom) within columns. Detected via the font’s WMode or text matrix analysis showing $90^\circ/270^\circ$ rotation combined with CJK characters.

6 Subset Font Problem

6.1 6.1 Problem Statement

This is the single hardest engineering challenge in semantic PDF editing.

When a PDF is generated, most producers subset-embed fonts: they include only the glyphs actually used in the document. A font that normally contains 2000+ glyphs may be embedded with only 50-200 glyphs. This reduces file size dramatically but creates a fundamental problem for editing.

When a semantic edit replaces text — for example, changing “SANKET” to “SMITH” — the character ‘M’ (U+004D) might not exist in the embedded subset font.

6.2 6.2 CID Mapping Deep Dive

In a CID-keyed font (Type 0 with Identity-H encoding), the mapping chain is:

```

Byte sequence in content stream
↓
CID (Character ID) - a 16-bit integer
↓ [via ToUnicode CMap]
Unicode code point
↓ [via font program's cmap table]
GID (Glyph ID) - internal to the font program
↓ [via font program's glyf/CFF table]
Glyph outline (vector data)
↓ [via PDF rendering engine]
Pixel on page

```

To add a new character to a subset font, we must:

1. Find a font program that contains the desired glyph (the original full font, or a compatible substitute)
2. Extract the glyph outline data
3. Insert it into the embedded subset font's internal tables
4. Update all cross-references: `loca`, `hmtx`, `cmap`, `maxp`, `post`, `OS/2`
5. Assign a new CID (or reuse an unused one)
6. Update the ToUnicode CMap to include the new CID→Unicode mapping
7. Update the font descriptor's glyph width array

6.3 6.3 Font Expansion Strategy (FontTools)

The most viable approach uses `fontTools`, the Python library for font manipulation:

6.3.1 6.3.1 Algorithm: Subset Font Expansion

```

from fontTools.ttLib import TTFont
from fontTools.subset import Subsetter

def expand_subset_font(embedded_font_bytes, required_characters):
    """
    Add glyphs for `required_characters` to the embedded subset font.

    Returns: (modified_font_bytes, new_cid_mapping)
    """
    font = TTFont()

```

```

font.importXML_or_Binary(embedded_font_bytes)

# Step 1: Identify which required characters are missing
cmap = font.getBestCmap()
missing = [c for c in required_characters
           if ord(c) not in cmap]

if not missing:
    return embedded_font_bytes, {}

# Step 2: Locate the original full font
full_font_path = locate_full_font(font)
if full_font_path is None:
    raise FontNotFoundError(f"Cannot find full font for {font}")

full_font = TTFont(full_font_path)

# Step 3: Extract glyphs for missing characters
full_cmap = full_font.getBestCmap()
glyph_map = {} # unicode → glyph_name in full font

for char in missing:
    glyph_name = full_cmap.get(ord(char))
    if glyph_name is None:
        raise GlyphNotFoundError(f"Glyph not found for U+{ord(char):04X}")
    glyph_map[ord(char)] = glyph_name

# Step 4: Copy glyph outlines and metrics
glyph_table = font['glyf'] if 'glyf' in font else font['CFF ']
full_glyph_table = full_font['glyf'] if 'glyf' in full_font else full_font['CFF ']

for unicode_val, glyph_name in glyph_map.items():
    # Copy glyph outline
    new_glyph_name = f"uni{unicode_val:04X}"
    glyph_table[new_glyph_name] = full_glyph_table[glyph_name]

    # Copy horizontal metrics
    font['hmtx'][new_glyph_name] = full_font['hmtx'][glyph_name]

    # Update cmap
    font['cmap'].tables[0].cmap[unicode_val] = new_glyph_name

# Step 5: Update maxp (glyph count)
font['maxp'].numGlyphs = len(font.getGlyphOrder())

# Step 6: Recalculate checksums
# fontTools handles this on save

return font.write(), glyph_map

```

6.3.2 Locating the Full Font

This is the hardest sub-problem. Strategies in order of preference:

1. **Font name matching:** Extract the original font name from the FontDescriptor's `FontName` entry, search local system fonts
2. **Font fingerprint matching:** Hash the embedded font's `name` table, `OS/2` table metrics, and `PANOSE` data; match against a font database

3. **Google Fonts / system fallback:** For common fonts (Helvetica, Times, Courier), use standard metric-compatible replacements
4. **User-provided font:** Accept a font file path as API parameter
5. **Glyph synthesis:** As an absolute last resort, synthesize the missing glyph from the font's existing glyphs (e.g., 'M' could be constructed from 'N' + extra strokes — this is research-grade, not production-ready)

6.4 6.4 Why Most Editors Avoid This Problem

Commercial PDF editors avoid the subset font problem through three strategies:

1. **Full font re-embedding:** Replace the subset font with the complete font. This increases file size by 1-5 MB per font. Acceptable for desktop software, unacceptable for an API processing millions of documents.
2. **Font substitution:** Replace the font entirely with a similar one (Helvetica → Arial). This changes the visual appearance.
3. **Fallback overlay:** Draw replacement text as a new content stream element with a separate font, positioned to cover the original. This is fragile — it breaks when the text length changes, when the font metrics differ, or when the page is reprinted at a different resolution.

None of these strategies preserves fidelity while maintaining file size. The hybrid approach — expanding subset fonts on demand, caching expanded versions, and falling back to overlay when the full font is unavailable — is the recommended strategy.

6.5 6.5 Hybrid Strategy (Recommended)

```
def edit_text_with_font_handling(entity, new_value, pdf):
    original_font = entity.primary_font()

    # Strategy A: Direct replacement (characters already in font)
    if all_chars_in_font(new_value, original_font):
        return direct_replace(entity, new_value, pdf)

    # Strategy B: Expand subset font
    try:
        expanded_font = expand_subset_font(original_font, new_value)
        pdf.replace_font(entity.font_ref, expanded_font)
        return direct_replace(entity, new_value, pdf)
    except FontNotFoundError:
        pass

    # Strategy C: Overlay with fallback font
    fallback_font = select_metric_compatible_font(original_font)
    return overlay_replace(entity, new_value, pdf, fallback_font)
```

7 Semantic Layer

7.1 7.1 Document Object Graph

The document object graph is the bridge between semantic understanding and PDF primitives.

```
Document
+-- Page[0]
|   +-- TextBlock "Experience"
|   |   +-- Entity: Person.Name → "SANKET"
|   |   +-- Entity: Date.Range → "2019-2023"
|   |   +-- Entity: Company.Name → "Acme Corp"
|   +-- TextBlock "Education"
|       +-- Entity: Institution.Name → "MIT"
|       +-- Entity: Date.Graduation → "2019"
+-- Page[1]
    +-- ...
```

7.2 7.2 Entity Definition Schema

```
Entity {
    id: string                // UUID
    type: EntityType          // Enum
    value: string             // Current text value
    confidence: float         // 0.0 to 1.0
    page_id: int
    text_nodes: list[TextNodeRef] // Back-references
}

TextNodeRef {
    node_id: string
    fragment_index: int
    start_glyph: int
    end_glyph: int
    pdf_objects: list[PDFObjectRef] // Content stream, operator index
}

PDFObjectRef {
    stream_ref: string        // e.g. "31 0 R"
    operator_index: int       // Position within tokenized stream
    operand_index: int        // Which operand in Tj/TJ
    glyph_offset: int         // Offset within the string
}
```

7.3 7.3 Entity Types Catalog

Entity Type	Description	Example Values
Person.Name	A person's full name	"Sanket Kumar"
Person.Name.First	First/given name	"Sanket"
Person.Name.Last	Last/family name	"Kumar"
Company.Name	Organization name	"Acme Corporation"
Address.Street	Street address	"123 Main St"
Address.City	City name	"San Francisco"
Address.State	State/province	"California"

Entity Type	Description	Example Values
Address.Zip	Postal code	"94105"
Address.Country	Country name	"United States"
Date.Full	Full date string	"June 2026"
Date.Year	Year only	"2026"
Date.Range	Date range	"2019-2023"
Invoice.Number	Invoice identifier	"INV-2026-0042"
Invoice.Amount	Monetary amount	"\$1,234.56"
Contract.Party	Legal party name	"Acme Corp, Inc."
Phone.Number	Phone number	"+1 (415) 555-0123"
Email.Address	Email address	"sanket@example.com"
URL	Web URL	"https://example.com"

7.4 7.4 Entity Extraction Pipeline

```
def extract_entities(document_graph):
    entities = []

    # Pass 1: Regex-based extraction
    for text_node in document_graph.walk_text_nodes():
        for entity_type, pattern in ENTITY_PATTERNS.items():
            for match in pattern.finditer(text_node.text):
                entities.append(Entity(
                    type=entity_type,
                    value=match.group(),
                    confidence=pattern_confidence(entity_type),
                    text_nodes=[text_node.create_ref(match.start(), match.end())]
                ))

    # Pass 2: Statistical/NER-based extraction (optional)
    for text_node in document_graph.walk_text_nodes():
        ner_results = named_entity_recognition(text_node.text)
        for entity in ner_results:
            entities.append(entity_from_ner(entity, text_node))

    # Pass 3: Deduplication and confidence refinement
    entities = deduplicate_entities(entities)
    entities = refine_confidence(entities, document_graph)

    return entities
```

7.5 7.5 Confidence Scoring

Entity confidence is computed from multiple signals:

```
def compute_confidence(entity, document_graph):
    score = 0.0
    signals = 0

    # Signal 1: Regex match
    if entity.matched_by_pattern:
        score += 0.7
        signals += 1

    # Signal 2: NER agreement
```

```
if entity.ner_type == entity.type:
    score += 0.2
    signals += 1

# Signal 3: Positional context
if entity.is_in_expected_position(document_graph):
    score += 0.1
    signals += 1

# Signal 4: Surrounding text context
if entity.has_context_keywords(document_graph):
    score += 0.1
    signals += 1

return score / max(signals, 1)
```


8 Semantic Editing Engine

8.1 Core Operation

The fundamental operation is:

```
replace_entity(type="Person.Name", old_value="SANKET", new_value="TEST")
```

Or, positionally:

```
replace_entity(entity_id="uuid-1234", new_value="TEST")
```

8.2 Edit Pipeline

8.2.1 Step 1: Entity Resolution

Resolve the entity specification to zero, one, or more entity instances. If `entity_id` is provided, resolve directly. If `type` and `old_value` are provided, search the entity catalog.

```
def resolve_entities(spec):
    if spec.entity_id:
        return [entity_catalog[spec.entity_id]]
    else:
        return [e for e in entity_catalog
                if e.type == spec.type and e.value == spec.old_value]
```

8.2.2 Step 2: PDF Object Resolution

For each matched entity, traverse its `text_nodes` to collect all `PDFObjectRef` instances. Group by content stream.

```
def collect_pdf_objects(entity):
    objects_by_stream = defaultdict(list)
    for text_node in entity.text_nodes:
        for obj_ref in text_node.pdf_objects:
            objects_by_stream[obj_ref.stream_ref].append(obj_ref)
    return objects_by_stream
```

8.2.3 Step 3: Glyph Validation

Before mutating, validate:

1. All glyphs required by `new_value` exist in the font (after expansion)
2. The new text fits within available space (width check)
3. The new text does not overflow the line or page
4. The new text does not collide with adjacent content

```
def validate_mutation(entity, new_value):
    font = entity.primary_font()
    required_width = 0
    for char in new_value:
        glyph_width = font.get_glyph_width(ord(char))
        required_width += glyph_width * font.size / 1000.0
```

```

available_width = entity.bounding_box().width

if required_width > available_width * 1.05:
    return MutationError(
        f"New text requires {required_width:.1f}pt, "
        f"but only {available_width:.1f}pt available"
    )

return MutationOK()

```

8.2.4 Step 4: Content Stream Mutation

Replace the original text strings in the content stream with the new text, adjusting kerning values as necessary.

```

def mutate_content_stream(stream_bytes, entity, new_value):
    tokenizer = PDFTokenizer(stream_bytes)
    output = bytearray()

    for token in tokenizer:
        for op_ref in entity.pdf_objects_for_stream(tokenizer.stream_ref):
            if token.operator_index == op_ref.operator_index:
                # This token is part of the entity to replace
                if token.op in (b"Tj", b"TJ"):
                    token = replace_text_in_operator(token, op_ref, new_value)
                output.extend(token.to_bytes())

    return bytes(output)

```

8.2.5 Step 5: PDF Structural Update

After content stream mutation:

1. Update the stream's `/Length` entry
2. If fonts were expanded, update the font file stream and font descriptor
3. Update the XRef table
4. Recompute the file trailer's `/Size` and byte offsets

8.2.6 Step 6: Validation

After saving, reopen the modified PDF and verify:

1. The file is structurally valid (no XRef corruption)
2. The modified text renders the new value
3. Unmodified text renders identically to the original
4. Page count is unchanged
5. File size change is within expected bounds

8.3 Edit Granularity Levels

Level	Operation	Example
Character	Change one character	Fix typo: "recieve" → "receive"
Word	Change one word	"Manager" → "Director"
Phrase	Change multi-word text	"Acme Corp" → "Globex Inc"

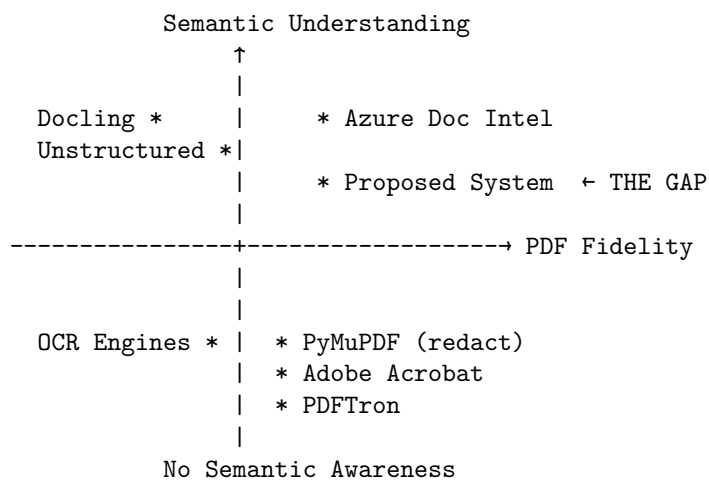
Level	Operation	Example
Entity	Change all occurrences of an entity type	All Person.Name → new value
Template	Fill a template with variable data	Resume template + JSON data → filled PDF
Batch	Apply same edit across multiple PDFs	Redact company name from 10,000 invoices

9 Competition Analysis

9.1 9.1 Technical Comparison Matrix

Feature	Adobe Acrobat	PyMuPDF	PDFTron/Able2Extract	Altera	Docling	Unstructured	Azure Doc Intel	Proposed
Text Extraction	[x]	[x]	[x]	[x]	[x]	[x]	[x]	[x]
Table Extraction	[x]	[]	[x]	Partial	[x]	[x]	[x]	[x]
Reading Order	[x]	Partial	[x]	[]	[x]	[x]	[x]	[x]
PDF Object Access	[x]	Partial	[x]	[x]	[]	[]	[]	[x]
Content Stream Editing	[x]	[x]	[x]	[x]	[]	[]	[]	[x]
Font Manipulation	[x]	[]	[x]	[x]	[]	[]	[]	[x]
Subset Font Expansion	[x]	[]	[x]	[]	[]	[]	[]	[x]
Semantic Entities	[]	[]	[]	[]	Partial	[x]	[x]	[x]
Entity→PDF Reverse Map	[]	[]	[]	[]	[]	[]	[]	[x]
Targeted Single-Field Edit	Partial	[]	Partial	Partial	[]	[]	[]	[x]
Fidelity Preservation	[x]	[x]	[x]	[x]	N/A	N/A	N/A	[x]
Automation API	IS API	Python	C++/Java/.NET	Java/.NET	Python	Python	REST	REST + Python
Open Source	[]	AGPL	[]	AGPL	MIT	Apache 2.0	[]	Apache 2.0
Batch Processing	Action Wizard	Manual	[x]	[x]	[x]	[x]	[x]	[x]
OCR	[x]	[]	[x]	[]	[]	[]	[x]	[x]
Fall-back								
XObject Support	[x]	[x]	[x]	[x]	[]	[]	[]	[x]
Multi-Column Layout	[x]	[]	[x]	[]	Partial	[x]	[x]	[x]

9.2 9.2 Market Positioning



The proposed system occupies the currently empty quadrant: **high semantic understanding + high PDF fidelity preservation.**

10 Failure Modes

10.1 10.1 Missing Glyphs

Condition	Detection	Mitigation
Replacement char not in font Full font unavailable	Pre-edit glyph check FontNotFoundError from locate_full_font()	Font expansion (Strategy B) Overlay with metric-compatible fallback (Strategy C)
Glyph in font but metrics missing	Missing hmtx entry	Use default width (FontDescriptor.AvgWidth or 1000)
CID collision with existing glyph	CID already mapped to different Unicode	Assign new CID at end of range

10.2 10.2 Broken or Missing ToUnicode CMaps

Many PDFs, especially older ones and those from certain generators, have no ToUnicode CMap or have a CMap that maps only a subset of glyphs.

Condition	Detection	Mitigation
No ToUnicode CMap	font.ToUnicode is None	Fall through to Encoding → glyph name resolution
ToUnicode maps only subset	Some CIDs return None from lookup	Per-glyph fallback: try Encoding, then cmap table, then AGL
Corrupted CMap stream	Parse error during CMap reading	Discard CMap, use fallback chain
Encoding maps to non-standard names	Glyph name not in AGL	Font program cmap table (TrueType) or heuristic matching

10.3 10.3 Image-Only PDFs (Scanned Documents)

These PDFs contain no text operators at all. Each page is a single image.

Detection: Zero text fragments after content stream parsing.

Mitigation: OCR pipeline fallback.

```
if len(page_fragments) == 0:
    image = rasterize_page(page)
    ocr_results = ocr_engine.recognize(image)
    # Overlay OCR text as invisible text layer
    # OR: create a new content stream with OCR text
    # Semantic layer operates on OCR output
    # Mutation layer must handle image-based PDFs differently
```

10.4 10.4 Rotated Text

Text rotated at arbitrary angles (not just 90° multiples) breaks the Y-clustering algorithm.

Detection: Text matrix Tm has non-zero b or non-zero c with both non-zero.

Mitigation: The clustering algorithm must detect rotation and transform coordinates to a normalized space before clustering.

```

if abs(rotation_deg) > 1.0:
    # Apply inverse rotation to glyph positions before clustering
    glyphs_normalized = [rotate_point(g, -rotation_deg) for g in glyphs]
    lines = cluster_by_y(glyphs_normalized, tolerance)
    # Map results back to original coordinates

```

10.5 10.5 Tables

Tables present unique challenges for both extraction and editing:

Challenge	Extraction Impact	Editing Impact
Ruling lines drawn separately from text	Cannot determine cell boundaries from text alone	Editing may cause text to cross cell boundaries
Alignment (left/center/right within cells)	Extracted text loses alignment info	Must preserve alignment when replacing
Merged cells	Text may span multiple logical cells	Replacement text might not fit merged cell width
Numeric formatting	Decimal alignment is lost	Replacing numbers may break alignment

Mitigation: Detect tables by identifying horizontal/vertical ruling lines near text blocks. Build a table model that constrains edits to cell boundaries.

10.6 10.6 Watermarks

Watermarks are typically large, rotated, low-opacity text rendered in a Form XObject that is repeated on every page.

Detection: XObject with high text size (>40pt), non-zero rotation, and ExtGState with low opacity.

Mitigation: Mark watermark text as non-editable by default. If editing is required, treat as a special entity type with full-page replacement semantics.

10.7 10.7 Headers and Footers

Headers and footers are often Form XObjects shared across pages.

Detection: XObject referenced by multiple pages, positioned at page boundaries (± 72 pt from top/bottom edge).

Mitigation: Edit the XObject once, and all pages are updated automatically. This is actually a benefit of the architecture — XObject-based page furniture is edited in one place.

10.8 10.8 Multi-Column Layouts

Two-column and three-column layouts break the simple Y-clustering approach because two text blocks may share the same Y coordinate but belong to different columns.

Detection: Multiple text blocks with overlapping Y ranges but non-overlapping X ranges.

Mitigation: Cluster by X first (column detection), then by Y within each column.

```
def detect_columns(page_glyphs):
    # Project glyphs onto X-axis
    x_histogram = build_x_histogram(page_glyphs)
    # Find valleys (gaps between columns)
    columns = find_histogram_valleys(x_histogram)
    # Assign glyphs to columns
    for glyph in page_glyphs:
        glyph.column = assign_column(glyph.x, columns)
    # Y-cluster within each column independently
    return columns
```


11 Evaluation Framework

11.1 Core Metrics

11.1.1 Text Reconstruction Accuracy (TRA)

Measures how accurately the system reconstructs the logical text from fragmented PDF operators.

$$\text{TRA} = |\text{correctly_reconstructed_tokens}| / |\text{total_tokens}|$$

where a token is a word-level text unit. A reconstruction is “correct” if:

- The characters are in the correct order
- The word boundaries are in the correct positions
- No characters from adjacent text blocks are merged

Target: >99% on digitally generated PDFs, >95% on scanned+OCR PDFs.

11.1.2 Entity Recall (ER)

Measures how many of the ground-truth entities the system correctly identifies.

$$\text{ER} = |\text{correctly_extracted_entities}| / |\text{ground_truth_entities}|$$

where “correctly extracted” requires:

- Correct entity type
- Correct text value
- Correct bounding box (IoU > 0.9 with ground truth)

Target: >95% for regex-extractable entities (names, dates, emails), >85% for NER-based entities.

11.1.3 Layout Preservation Score (LPS)

Measures how well the system preserves the original document layout after editing.

$$\text{LPS} = 1.0 - (|\text{modified_pixels}| / |\text{total_pixels}|)$$

where `modified_pixels` counts pixels that differ between the original and edited PDF (outside the edited entity’s bounding box).

Target: LPS > 0.999 (fewer than 0.1% of pixels outside the edit region should change).

11.1.4 Edit Success Rate (ESR)

The percentage of attempted entity edits that succeed without manual intervention.

$$\text{ESR} = |\text{successful_edits}| / |\text{attempted_edits}|$$

An edit is “successful” if:

- The target text is replaced correctly
- The file is structurally valid after editing
- No visual artifacts are introduced outside the edit region
- The edited entity remains extractable with correct value

Target: >90% on digitally generated PDFs.

11.1.5 11.1.5 Font Preservation Score (FPS)

Measures whether the edited text uses the same font as the original (or a metric-compatible expansion).

```
FPS = 1.0 if expanded subset font was used
      0.8 if metric-compatible fallback was used
      0.5 if generic fallback was used
      0.0 if font was substituted entirely
```

Target: Average FPS > 0.85 across all edits.

11.2 11.2 Benchmark Dataset Design

A proper evaluation requires a diverse benchmark dataset. Proposed composition:

Category	Count	Description
Modern Word PDFs	100	PDFs generated by Word 2016+, various templates
LaTeX PDFs	100	pdflatex, xelatex, lualatex output, various document classes
InDesign PDFs	50	Professionally designed documents
Browser PDFs	50	Chrome/Firefox/Safari print-to-PDF
Legacy PDFs	50	Older PDF versions (1.2-1.4), various generators
Scanned PDFs	50	Image-only PDFs requiring OCR
Forms	50	Fillable PDF forms (AcroForms)
Complex Layouts	50	Multi-column, tables, sidebars

Per document: - Ground truth text (word-level, with positions) - Ground truth entities (type, value, bounding box) - Edit specifications (entity to modify, new value) - Expected post-edit state (for automated verification)

12 Engineering Roadmap

12.1 Phase 1: Reconstruction Engine (Months 1-3)

Goal: Parse arbitrary PDFs and produce a complete, correct glyph-to-Unicode mapping with positions.

12.1.1 Deliverables:

- **Content stream tokenizer:** Full support for all text operators (BT/ET, Tf, Tm, Td, TD, Tj, TJ, ', ", Ts, Tc, Tw, Tz, Tr)
- **Font graph builder:** ToUnicode CMap parser, Encoding parser, TrueType cmap parser
- **PDF object traversal:** Page tree walker, XObject recursion, resource inheritance
- **Glyph position calculator:** Text matrix transformer with rotation support
- **Output:** JSON glyph manifest per page

12.1.2 Key Risks:

- Non-standard CMaps from obscure generators
- Corrupted content streams requiring graceful degradation
- Type 3 (user-defined) fonts with no extractable Unicode mapping

12.2 Phase 2: Layout Graph (Months 3-5)

Goal: Reconstruct logical document structure from glyph positions.

12.2.1 Deliverables:

- **Y-clustering algorithm** with rotation compensation
- **Token clustering** (word formation) with gap threshold tuning
- **Line merging** and reading order assignment
- **Block detection** (paragraphs, headings)
- **Table detection** using ruling line analysis
- **Column detection** using X-projection histogram
- **Header/footer detection** via page boundary proximity + repetition
- **Output:** Layout graph as hierarchical JSON

12.2.2 Key Risks:

- False line merges due to subscript/superscript near line boundaries
- Incorrect reading order in heavily designed layouts

12.3 Phase 3: Semantic Layer + Editing (Months 5-9)

Goal: Build the entity extraction pipeline and the mutation engine.

12.3.1 Deliverables:

- **Entity extraction:** Regex patterns + optional NER integration
- **Entity catalog:** Bidirectional entity <-> PDF object mapping
- **Mutation engine:** Direct replace, font expansion, overlay fallback
- **Subset font expansion:** FontTools-based glyph injection
- **Edit validation:** Pre-edit checks + post-edit verification
- **API surface:** REST API + Python SDK
- **Output:** Editable semantic document model

12.3.2 Key Risks:

- Subset font expansion failing on CFF (PostScript outline) fonts
- Content stream mutation corrupting stream structure
- XRef table regeneration edge cases

12.4 Phase 4: API Platform (Months 9-12)

Goal: Productionize the system as a developer platform.

12.4.1 Deliverables:

- **REST API:** CRUD for documents, entities, edits
- **Batch processing:** Asynchronous job queue for 10,000+ document batches
- **Webhook notifications:** Edit completion callbacks
- **SDKs:** Python, JavaScript, Java
- **Authentication:** API key management, rate limiting
- **Monitoring:** Edit success rates, latency percentiles, error budgets
- **Documentation:** API reference, tutorials, integration guides

12.5 Phase 5: Enterprise Workflows (Months 12-18)

Goal: Build high-value workflow applications on top of the platform.

12.5.1 Deliverables:

- **Resume/CV redaction:** Bulk anonymization for recruiting
- **Invoice editing:** Field-level corrections without regeneration
- **Contract management:** Party name updates, clause replacement
- **Compliance redaction:** PII removal with audit trail
- **Template filling:** JSON → PDF with font fidelity

13 Open Questions

13.1 13.1 Subset Font Expansion Limits

- What is the practical upper bound on glyph injection before the font's internal tables exceed PDF limits? The `loca` table (glyph location offsets) and `hmtx` table (horizontal metrics) grow linearly with glyph count. At what point does this become unacceptable?
- Can we share expanded font objects across multiple edits to the same document to avoid repeated expansion?
- Is there a viable glyph synthesis approach for fonts where the full original is truly unavailable?

13.2 13.2 Semantic Confidence Measurement

- How should we combine regex, NER, positional, and contextual signals into a single confidence score that is both interpretable and actionable?
- Should the system refuse to edit entities below a confidence threshold, or should it proceed with warnings?
- Can we learn confidence thresholds per document type (resume vs. invoice vs. academic paper)?

13.3 13.3 Layout Graph Generalization

- Can a single layout reconstruction algorithm handle the diversity of PDF generators (Word, LaTeX, InDesign, Chrome, Scribus, LibreOffice)?
- Or do we need generator-specific layout heuristics?
- How do we detect which generator produced a PDF to select the right heuristic?

13.4 13.4 OCR Integration Depth

- For scanned PDFs, should the OCR output be treated as a separate “virtual text layer” that overlays the image, or should we inject OCR text as new content stream operators?
- How do we handle OCR confidence at the word and character level in the mutation engine?
- Can OCR output achieve the same fidelity preservation guarantees as native text extraction?

13.5 13.5 Performance at Scale

- What is the processing throughput for a single PDF page (parsing + reconstruction + layout + entity extraction)? Target: < 500ms per page.
- Can the reconstruction be incremental (re-parse only modified streams) or must it be full-document?
- What is the memory profile for a 100-page, 50MB PDF?

13.6 13.6 Security Considerations

- Does the content stream parser need to sandbox execution? (PDF streams are not executable, but malicious streams could attempt buffer overflow attacks on naive parsers)
- Should edited documents carry a digital signature indicating they were transformed by this platform?
- How do we handle password-protected PDFs? Decrypt-once, edit, re-encrypt? Or refuse?

This document is RFC-001 for the Semantic PDF Transformation Platform.

Feedback, criticism, and counter-proposals are explicitly solicited.

Contact: architecture@semantic-pdf.dev